

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



THIẾT KẾ VI MẠCH (CO3098)

Bài Thực Hành 1

DIGITAL DESIGN SIMULATION USING XCELIUM

Lớp: L04 - Nhóm: 08 - Học Kỳ HK252
Ngày nộp: 30/03/2026

GVHD: Thầy Phạm Kiều Nhật Anh

Sinh Viên	MSSV
Phạm Công Võ	2313946
Phạm Minh Đức	2311079
Nguyễn Trần Như Phước	2212719
Nguyễn Thanh Tùng	2112711

TP.Hồ Chí Minh, Tháng 03/2026

Mục lục

1	Introduction	2
1.1	Mục tiêu Lab	2
1.2	Công cụ sử dụng	2
1.3	Tổng quan thiết kế - Bound Flasher	2
2	Design Specification	3
2.1	Mô tả chức năng	3
2.2	Interface - Bảng tín hiệu	5
2.3	Bảng mô tả hoạt động các Phase	5
3	RTL Implementation	7
3.1	Sơ đồ khối hệ thống	7
3.2	Máy trạng thái (FSM)	8
3.3	RTL Code	9
4	Testbench	12
4.1	Tổng quan Testbench	12
4.2	Cấu trúc Testbench	12
4.3	Mô tả các Test Case	14
4.4	Chi tiết từng Test Case	15
5	Kết luận	19
5.1	Tổng kết Lab	19
5.2	Kết quả đạt được	19
5.3	Những điểm đã học được	19

1 Introduction

1.1 Mục tiêu Lab

Lab 1 nhằm trang bị cho sinh viên nền tảng thực hành trong quy trình kiểm tra và xác minh thiết kế vi mạch số ở mức RTL (*Register Transfer Level*). Cụ thể, sau khi hoàn thành Lab, sinh viên có khả năng:

- Xây dựng môi trường mô phỏng RTL chuẩn công nghiệp trên nền tảng **Cadence Xcelium**, bao gồm cấu trúc thư mục, thiết lập license và biên dịch thiết kế từ command-line.
- Viết **testbench** bằng Verilog để tạo stimulus và kiểm tra tính đúng đắn của thiết kế theo đặc tả kỹ thuật.
- Xuất và phân tích **waveform** sử dụng công cụ đồ họa **SimVision**, bao gồm kỹ thuật trace signal, duyệt source code và quan sát schematic trong quá trình debug.
- Hình thành tư duy kiểm tra có hệ thống thông qua việc xây dựng nhiều test case bao phủ các tình huống hoạt động bình thường và ngoại lệ của thiết kế.

1.2 Công cụ sử dụng

Công cụ	Phiên bản	Vai trò
Cadence Xcelium	2021	Trình biên dịch và thực thi mô phỏng HDL. Hỗ trợ Verilog/SystemVerilog, cho phép chạy simulation từ command-line với lệnh <code>xrun</code> .
SimVision	2021	Giao diện đồ họa tích hợp để xem waveform, duyệt source code (<i>Source Browser</i>) và trace logic qua <i>Schematic Tracer</i> .

Bảng 1: Công cụ sử dụng trong Lab 1

1.3 Tổng quan thiết kế - Bound Flasher

Thiết kế trong Lab 1 là module **Bound Flasher** - một mạch điều khiển tuần tự điều khiển dãy 16 đèn (`lamp[15:0]`) theo một chuỗi bật/tắt có điều kiện, được hiện thực bằng kiến trúc **Mealy/Moore FSM** trong Verilog RTL.

Dự án bao gồm hai thành phần chính:

File	Vai trò
<code>bound_flasher.v</code>	RTL design — mô tả hành vi phần cứng của module Bound Flasher theo kiến trúc FSM 9 trạng thái.
<code>bound_flasher_tb.v</code>	Testbench — tạo môi trường mô phỏng với 6 test case kiểm tra toàn diện các kịch bản hoạt động của thiết kế.

Bảng 2: Các file thiết kế trong Lab 1

2 Design Specification

2.1 Mô tả chức năng

Module **Bound Flasher** hiện thực hệ thống điều khiển 16 đèn LED (`lamp[15:0]`) hoạt động theo chuỗi tuần tự có điều kiện, dựa trên ba tín hiệu đầu vào: `clk`, `rst_n` và `flick`.

Mô tả tín hiệu điều khiển:

- **Clock** (`clk`): Cung cấp xung nhịp cho hệ thống. Các chuyển trạng thái được thực hiện tại **cạnh lên** (*positive edge*) của tín hiệu clock.
- **Reset** (`rst_n`): Tín hiệu reset tích cực thấp (*active-low*).
 - `rst_n = 0`: Hệ thống lập tức trở về trạng thái khởi tạo, toàn bộ đèn tắt.
 - `rst_n = 1`: Hệ thống hoạt động bình thường.
- **Flick** (`flick`): Tín hiệu điều khiển đặc biệt, có hai vai trò:
 - Kích hoạt chuỗi hoạt động khi hệ thống đang ở *Initial State*.
 - Kích hoạt cơ chế **kickback** tại các điểm nút trong quá trình bật đèn.

Chuỗi hoạt động:

Tại *Initial State*, toàn bộ đèn ở trạng thái **tắt**. Khi `flick` được kích hoạt (`flick = 1`), hệ thống bắt đầu chuỗi bật/tắt đèn theo thứ tự sau:

1. Bật dần các đèn từ `lamp[0]` đến `lamp[5]`.
2. Tắt dần các đèn từ `lamp[5]` về `lamp[0]`.
3. Bật dần các đèn từ `lamp[0]` đến `lamp[10]`.
4. Tắt dần các đèn từ `lamp[10]` về `lamp[5]`.
5. Bật dần các đèn từ `lamp[5]` đến `lamp[15]`.
6. Tắt dần các đèn từ `lamp[15]` về `lamp[0]`, trở về *Initial State*.

Điều kiện Kickback:

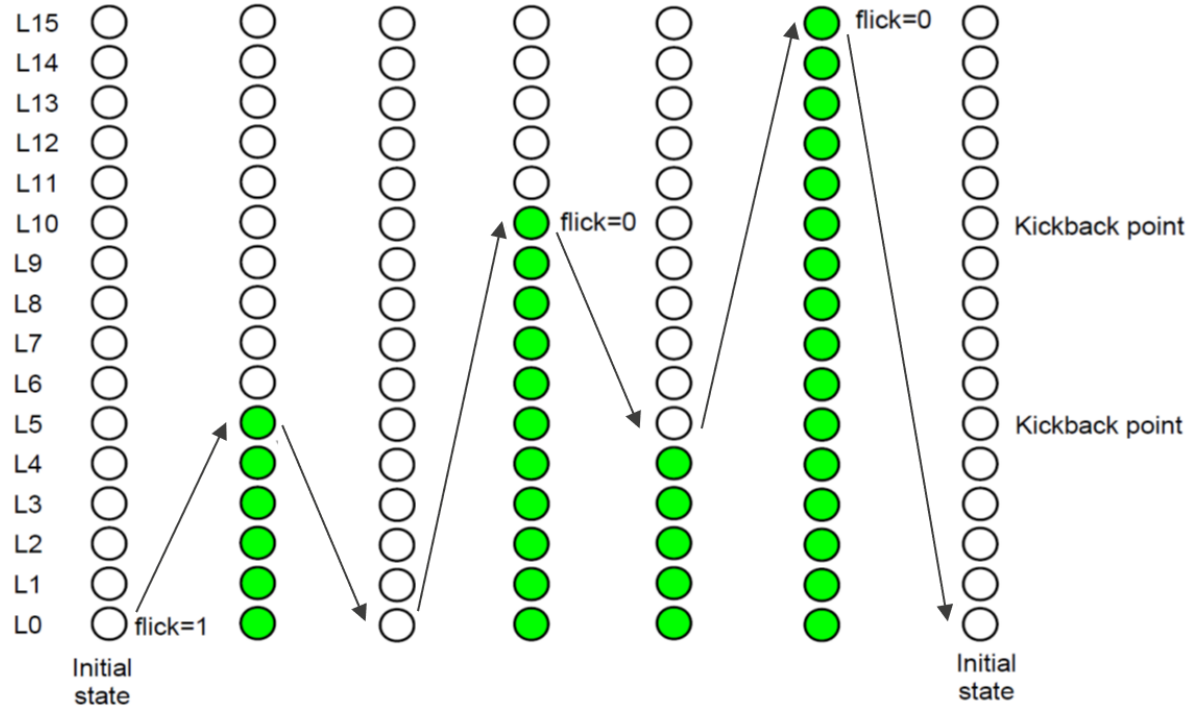
Tại các điểm nút **kickback point** (`lamp[5]` và `lamp[10]`), nếu tín hiệu `flick` đang **Tích Cực** trong quá trình **Bật Dần**, hệ thống sẽ **tắt dần về mức cực tiểu của giai đoạn trước**, sau đó tiếp tục chuỗi hoạt động như mô tả ở trên. Các ràng buộc quan trọng:

- Kickback chỉ áp dụng trong các phase **bật đèn dần** (*ON gradually*), ngoại trừ giai đoạn đầu tiên (bước 1: `lamp[0] → lamp[5]`).
- Tại điểm nút `lamp[5]` trong bước 3: nếu `flick = 1`, đèn sẽ tắt dần về `lamp[0]` rồi tiếp tục bật lại từ đầu bước 3.
- Tại điểm nút `lamp[10]` trong bước 3: nếu `flick = 1`, đèn sẽ tắt dần về `lamp[0]` rồi tiếp tục bật lại từ đầu bước 3.
- Tại điểm nút `lamp[10]` trong bước 5: nếu `flick = 1`, đèn sẽ tắt dần về `lamp[5]` rồi tiếp tục bật lại từ đầu bước 5.

- Nếu $flick = 0$ tại các điểm nút, hệ thống tiếp tục chuỗi hoạt động bình thường không có kickback.

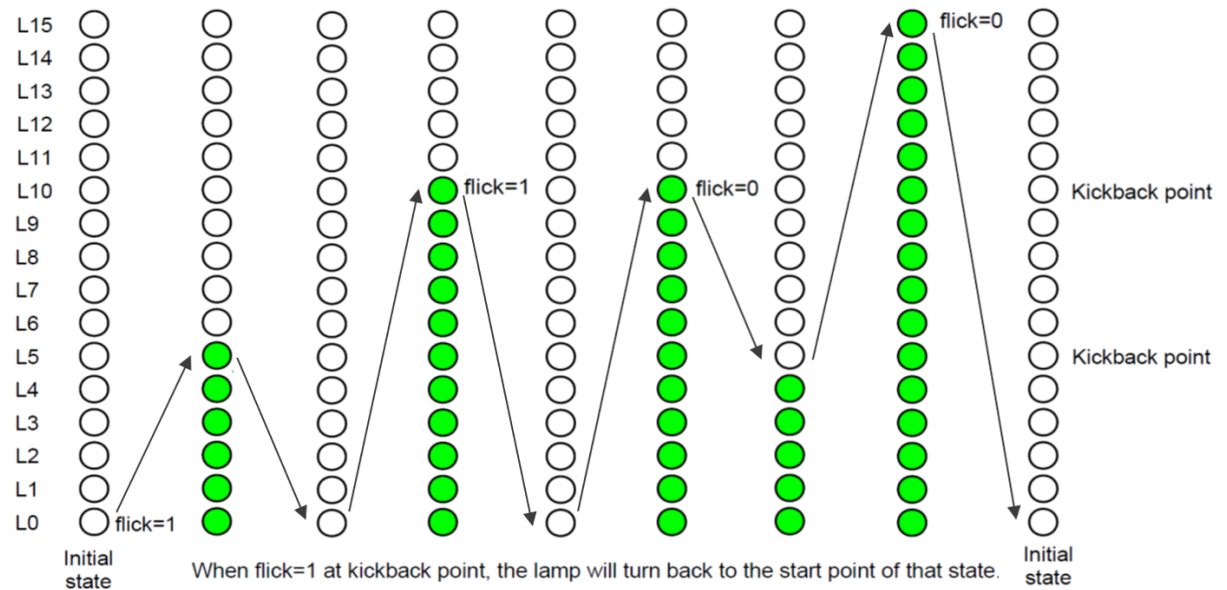
Ví dụ minh họa hoạt động

Khi $flick = 0$ tại điểm kickback $lamp[5]$, hệ thống **không kích hoạt cơ chế kickback** mà tiếp tục chuỗi hoạt động bình thường theo đúng thứ tự đã mô tả.



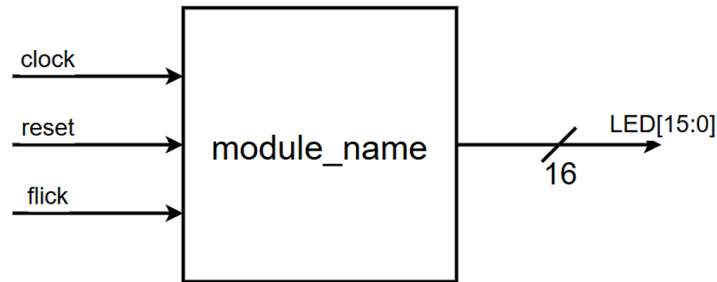
Hình 1: Hoạt động bình thường khi $flick = 0$ tại điểm kickback

Khi $flick = 1$ tại điểm kickback $lamp[10]$, hệ thống **kích hoạt cơ chế kickback** - đèn sẽ tắt dần về mức cực tiểu của giai đoạn trước, sau đó tiếp tục chuỗi hoạt động bình thường.



Hình 2: Hoạt động kickback khi $flick = 1$ tại điểm kickback LED[10]

2.2 Interface - Bảng tín hiệu



Hình 3: Sơ đồ hệ thống Bound Flasher

Tín hiệu	Hướng	Độ rộng	Mô tả chi tiết
clk	Input	1-bit	Xung nhịp đồng hồ hệ thống. Toàn bộ logic tuần tự cập nhật tại cạnh lên (<i>posedge</i>).
rst_n	Input	1-bit	Reset tích cực thấp (<i>active-low</i>). Khi rst_n = 0, hệ thống lập tức trở về trạng thái INIT và tắt toàn bộ đèn.
flick	Input	1-bit	Tín hiệu kích hoạt đa chức năng: khởi động chuỗi khi ở trạng thái INIT, đồng thời là điều kiện kích hoạt kickback tại các điểm nút.
lamp[15:0]	Output	16-bit	Thanh ghi đầu ra trạng thái 16 đèn. Mỗi bit tương ứng một đèn: 1 = sáng, 0 = tắt. Cập nhật đồng bộ theo clk.

Bảng 3: Bảng tín hiệu giao tiếp của module *bound_flasher*

2.3 Bảng mô tả hoạt động các Phase

Chuỗi hoạt động hoàn chỉnh của Bound Flasher được tổ chức thành 9 trạng thái FSM, phân chia thành các giai đoạn bật/tắt đèn tuần tự. Tại mỗi chu kỳ clock, hệ thống thêm hoặc bớt đúng một đèn thông qua phép toán dịch bit:

- Bật thêm một đèn: $lamp \leftarrow (lamp \ll 1)$
- Tắt bớt một đèn: $lamp \gg 1$

Mỗi cycle tương ứng một cạnh lên của clk. Mỗi bước trong chuỗi bật/tắt đèn thêm hoặc bớt đúng 1 đèn mỗi cycle.

Tổng thời gian một chu kỳ hoàn chỉnh là **62 clock cycles**, trong trường hợp **không xảy ra kickback** (tức là tín hiệu flick luôn bằng 0 tại các thời điểm kickback).

State	Giai đoạn	Hành động	Phạm vi đèn	Cycles	Điều kiện chuyển tiếp
INIT	Khởi tạo	Toàn bộ đèn tắt, chờ kích hoạt	0x0000	—	flick=1 → S1
S1	Phase 1: ON	Bật dần lamp[0] → lamp[5]	0x0001 → 0x003F	6	lamp=0x003F → S2
S2	Phase 2: OFF	Tắt dần lamp[5] → lamp[0]	0x003F → 0x0000	6	lamp=0x0000 → S3
S3	Phase 3: ON	Bật dần lamp[0] → lamp[10]	0x0001 → 0x07FF	11	lamp=0x07FF → S4; kickback → S3K
S3K	Kickback S3	Tắt dần về lamp[0], khởi động lại S3	→ 0x0000	6–11	lamp=0x0000 → S3
S4	Phase 4: OFF	Tắt dần lamp[10] → lamp[5]	0x07FF → 0x003F	6	lamp=0x003F → S5
S5	Phase 5: ON	Bật dần lamp[5] → lamp[15]	0x003F → 0xFFFF	11	lamp=0xFFFF → S6; kickback → S5K
S5K	Kickback S5	Tắt dần về lamp[5], khởi động lại S5	→ 0x003F	6	lamp=0x003F → S5
S6	Phase 6: OFF	Tắt hoàn toàn lamp[15] → lamp[0]	0xFFFF → 0x0000	16	lamp=0x0000 → INIT

Bảng 4: Bảng mô tả 9 trạng thái FSM của Bound Flasher

Điều kiện Kickback

Kickback là cơ chế khi tín hiệu flick xuất hiện tại các **kickback point**, hệ thống sẽ tạm dừng quá trình bật đèn, đảo chiều (tắt dần về mức tối thiểu của giai đoạn trước), rồi tiếp tục lại chuỗi ban đầu.

#	Điểm nút	State	lamp	Điều kiện	Hành động	Quay về
1	lamp[5]	S3 (ON: L0→L10)	0x003F	flick=1	Tắt dần về lamp[0]	Khởi động lại S3 từ 0x0001
2	lamp[10]	S3 (ON: L0→L10)	0x07FF	flick=1	Tắt dần về lamp[0]	Khởi động lại S3 từ 0x0001
3	lamp[10]	S5 (ON: L5→L15)	0x07FF	flick=1	Tắt dần về lamp[5]	Khởi động lại S5 từ 0x003F

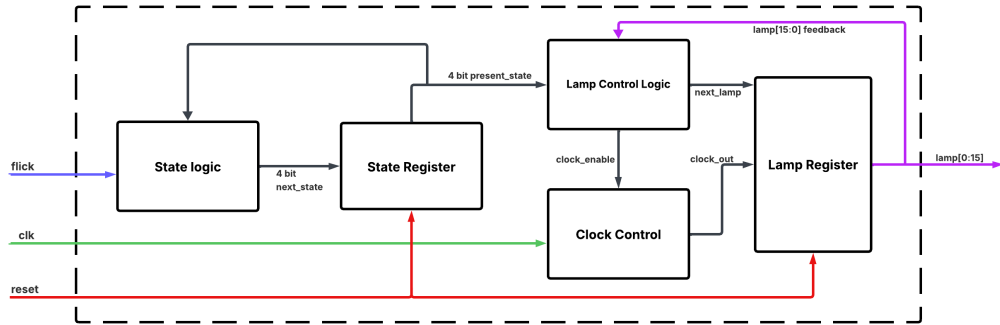
Bảng 5: Bảng điều kiện kickback của Bound Flasher

- **Ràng buộc 1 - Chỉ áp dụng trong phase bật đèn (ON):** Kickback không được kiểm tra trong các giai đoạn tắt đèn (S2, S4, S6, S3K, S5K). Thiết kế chỉ sample tín hiệu flick tại các điểm nút trong S3 và S5.
- **Ràng buộc 2 - Miễn trừ Phase 1 (S1):** Giai đoạn bật đầu tiên (lamp[0] → lamp[5]) không có cơ chế kickback theo đặc tả. Tín hiệu flick trong S1 bị bỏ qua hoàn toàn.

- **Ràng buộc 3 - Kickback có thể lặp nhiều lần:** Nếu flick vẫn tích cực sau mỗi lần kickback, hệ thống có thể lặp lại cơ chế này liên tục tại cùng một điểm nút - đây là hành vi cần được kiểm tra bằng test case riêng biệt.

3 RTL Implementation

3.1 Sơ đồ khối hệ thống



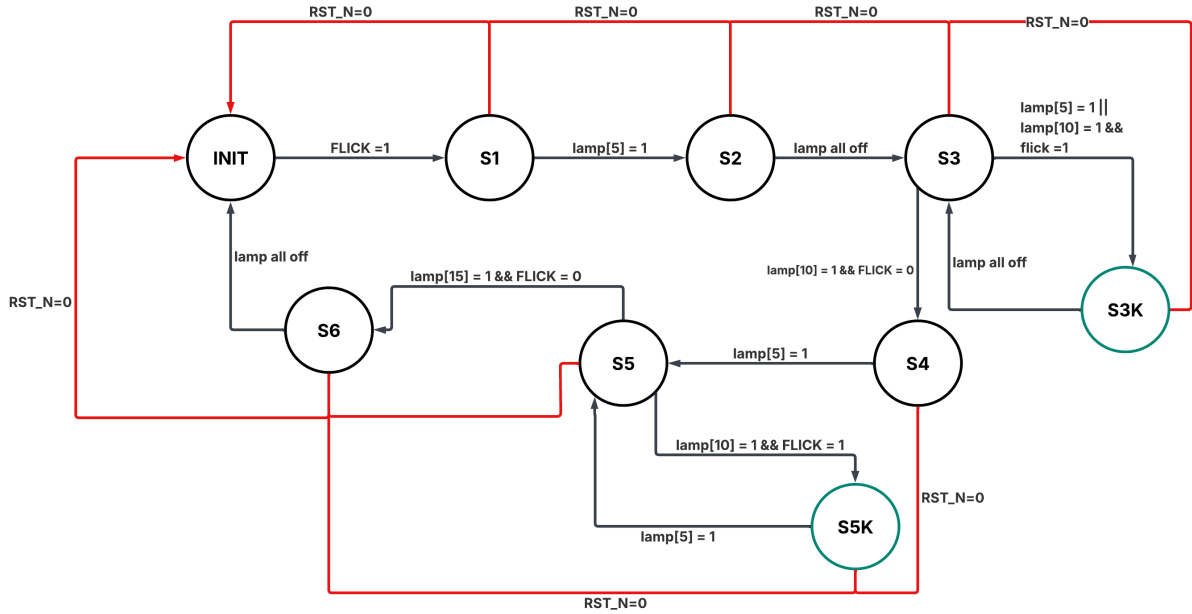
Hình 4: Sơ đồ khối của module *bound_flasher*

Module	Signal	Mô tả
State Logic	input flick [3:0] input present_state [15:0] input lamp [3:0] output next_state	Khối tổ hợp quyết định next_state dựa trên present_state , lamp và tín hiệu flick . Bao gồm logic phát hiện <i>kickback</i> tại các trạng thái S3 và S5.
State Register	input clk input rst_n [3:0] input next_state [3:0] output present_state	Thanh ghi tuần tự lưu trữ present_state , cập nhật từ next_state tại mỗi <i>posedge</i> clk . Reset về INIT khi rst_n = 0.
Lamp Control Logic	[3:0] input present_state [15:0] input lamp [15:0] output next_lamp	Khối tổ hợp tính toán next_lamp theo trạng thái: bật thêm ($\text{lamp} \mid (\text{lamp} \ll 1)$), tắt bớt ($\text{lamp} \gg 1$), hoặc giữ nguyên tại các điểm <i>kickback</i> . Nhận phản hồi lamp từ Lamp Register.
Clock Control	input clk input clock_enable output clock_out	Khối điều khiển xung clock cấp cho Lamp Register thông qua tín hiệu clock_enable , đảm bảo đèn chỉ cập nhật tại các thời điểm hợp lệ.
Lamp Register	input clock_out input rst_n [15:0] input next_lamp [15:0] output lamp[0:15]	Thanh ghi tuần tự lưu trữ giá trị lamp , cập nhật đồng bộ theo clock_out . Reset về 0x0000 khi rst_n = 0. Đầu ra phản hồi về State Logic và Lamp Control Logic.

3.2 Máy trạng thái (FSM)

Module **Bound Flasher** được hiện thực theo kiến trúc **Two-Always FSM** với **9 trạng thái**, phân chia thành 6 giai đoạn bật/tắt đèn tuần tự và 2 trạng thái kickback (S3K, S5K). Các mũi tên **đỏ** thể hiện reset toàn cục - khi `rst_n = 0`, hệ thống lập tức trở về INIT tại bất kỳ trạng thái nào.

Sơ đồ trạng thái



Hình 5: Sơ đồ máy trạng thái của module *bound_flasher*

Mã hóa trạng thái

STT	Trạng thái	Mã hóa	Ý nghĩa
0	INIT	4'd0	Khởi tạo, toàn bộ đèn tắt, chờ flick = 1
1	S1	4'd1	Phase 1 ON bật dần lamp[0] → lamp[5]
2	S2	4'd2	Phase 2 OFF - tắt dần lamp[5] → lamp[0]
3	S3	4'd3	Phase 3 ON - bật dần lamp[0] → lamp[10]
4	S3K	4'd4	Kickback S3 - tắt dần về lamp[0], quay lại S3
5	S4	4'd5	Phase 4 OFF - tắt dần lamp[10] → lamp[5]
6	S5	4'd6	Phase 5 ON - bật dần lamp[5] → lamp[15]
7	S5K	4'd7	Kickback S5 - tắt dần về lamp[5], quay lại S5
8	S6	4'd8	Phase 6 OFF - tắt hoàn toàn về lamp[0], về INIT

Bảng 6: Bảng mã hóa trạng thái của máy FSM

Điều kiện chuyển trạng thái

Từ	Đến	Điều kiện chuyển
INIT	S1	flick = 1
S1	S2	lamp = 0x003F
S2	S3	lamp = 0x0000
S3	S3K	(lamp = 0x003F hoặc lamp = 0x07FF) và flick = 1
S3	S4	lamp = 0x07FF và flick = 0
S3K	S3	lamp = 0x0000 (sau khi tắt dần về lamp[0])
S4	S5	lamp = 0x003F
S5	S5K	lamp = 0x07FF và flick = 1
S5	S6	lamp = 0xFFFF và flick = 0
S5K	S5	lamp = 0x003F (sau khi tắt dần về lamp[5])
S6	INIT	lamp = 0x0000 (sau khi tắt hoàn toàn)

Bảng 7: Bảng điều kiện chuyển trạng thái của máy FSM

3.3 RTL Code

```
1 module bound_flasher (  
2     output reg [15:0] lamp,  
3     input wire      clk,  
4     input wire      rst_n,  
5     input wire      flick  
6 );  
7  
8     // State parameter declarations  
9     parameter INIT = 4'd0; // Initial state: wait for flick  
10    parameter S1  = 4'd1; // Phase 1: ON  lamp[0]  -> lamp[5]  
11    parameter S2  = 4'd2; // Phase 2: OFF lamp[5]  -> lamp[0]  
12    parameter S3  = 4'd3; // Phase 3: ON  lamp[0]  -> lamp[10]  
13    parameter S3K = 4'd4; // Kickback S3: OFF back to lamp[0]  
14    parameter S4  = 4'd5; // Phase 4: OFF lamp[10] -> lamp[5]  
15    parameter S5  = 4'd6; // Phase 5: ON  lamp[5]  -> lamp[15]  
16    parameter S5K = 4'd7; // Kickback S5: OFF back to lamp[5]  
17    parameter S6  = 4'd8; // Phase 6: OFF lamp[15] -> lamp[0]  
18  
19    reg [3:0] state, next_state;  
20    reg [15:0] next_lamp;  
21  
22    // Sequential block: update state and lamp at posedge clk  
23    always @(posedge clk or negedge rst_n) begin  
24        if (!rst_n) begin  
25            state <= INIT; // Reset to initial state  
26            lamp  <= 16'h0000; // Turn off all lamps  
27        end else begin
```

```
28     state <= next_state; // Update to next state
29     lamp <= next_lamp;   // Update lamp value
30 end
31 end
32
33 // Combinational block: compute next_state and next_lamp
34 always @(*) begin
35     next_state = state; // Default: hold current state
36     next_lamp  = lamp;   // Default: hold current lamp value
37
38     case (state)
39
40         // INIT: Wait for flick = 1 to start the sequence
41         INIT: begin
42             next_lamp = 16'h0000;
43             if (flick)
44                 next_state = S1;
45             end
46
47         // S1: Turn on lamp[0] to lamp[5] gradually, NO kickback check
48         S1: begin
49             if (lamp == 16'h0000)
50                 next_lamp = 16'h0001; // Turn on the first lamp
51             else if (lamp == 16'h003F)
52                 next_state = S2; // Reached lamp[5] -> go to S2
53             else
54                 next_lamp = lamp | (lamp << 1); // Turn on one more lamp
55             end
56
57         // S2: Turn off lamp[5] down to lamp[0] gradually
58         S2: begin
59             if (lamp == 16'h0001) begin
60                 next_lamp = 16'h0000; // Turn off the last lamp
61                 next_state = S3; // Go to S3
62             end else
63                 next_lamp = lamp >> 1; // Turn off one lamp
64             end
65
66         // S3: Turn on lamp[0] to lamp[10] gradually, check kickback at L5 and L10
67         S3: begin
68             if (lamp == 16'h0000)
69                 next_lamp = 16'h0001; // Turn on the first lamp
70             else if ((lamp == 16'h003F || lamp == 16'h07FF) && flick) begin
71                 // Kickback: at L5 or L10 with flick=1 -> enter S3K
72                 next_lamp = lamp; // Hold current lamp value
73                 next_state = S3K;
74             end else if (lamp == 16'h07FF)
75                 next_state = S4; // Reached lamp[10], no kickback -> go to S4
76             else
77                 next_lamp = lamp | (lamp << 1); // Turn on one more lamp
78             end
79
80         // S3K: Turn off lamps back to lamp[0], then restart S3
81         S3K: begin
82             if (lamp == 16'h0001) begin
83                 next_lamp = 16'h0000; // Turn off the last lamp
84                 next_state = S3; // Return to S3
```

```
85     end else
86         next_lamp = lamp >> 1;           // Turn off one lamp
87     end
88
89     // S4: Turn off lamp[10] down to lamp[5] gradually
90     S4: begin
91         if (lamp == 16'h003F)
92             next_state = S5;           // Reached lamp[5] -> go to S5
93         else
94             next_lamp = lamp >> 1;     // Turn off one lamp
95         end
96
97         // S5: Turn on lamp[5] to lamp[15] gradually, check kickback at L10
98         S5: begin
99             if (lamp == 16'h07FF && flick) begin
100                 // Kickback: at L10 with flick=1 -> enter S5K
101                 next_lamp = lamp;     // Hold current lamp value
102                 next_state = S5K;
103             end else if (lamp == 16'hFFFF)
104                 next_state = S6;     // Reached lamp[15] -> go to S6
105             else
106                 next_lamp = lamp | (lamp << 1); // Turn on one more lamp
107             end
108
109             // S5K: Turn off lamps back to lamp[5], then restart S5
110             S5K: begin
111                 if (lamp == 16'h003F) begin
112                     next_lamp = lamp; // Hold 0x003F as starting point of S5
113                     next_state = S5;  // Return to S5
114                 end else
115                     next_lamp = lamp >> 1; // Turn off one lamp
116                 end
117
118                 // S6: Turn off all lamps from lamp[15] down to lamp[0], return to INIT
119                 S6: begin
120                     if (lamp == 16'h0001) begin
121                         next_lamp = 16'h0000; // Turn off the last lamp
122                         next_state = INIT;    // Return to initial state
123                     end else
124                         next_lamp = lamp >> 1; // Turn off one lamp
125                     end
126
127                     // Default: handle undefined states
128                     default: begin
129                         next_state = INIT;
130                         next_lamp = 16'h0000;
131                     end
132
133                 endcase
134             end
135
136     endmodule
```

4 Testbench

4.1 Tổng quan Testbench

Testbench được viết bằng Verilog, tổ chức thành **6 test case** độc lập, mỗi test case kiểm tra một kịch bản hoạt động cụ thể của module `bound_flasher`. Cấu trúc tổng quát gồm ba phần chính:

- **Khai báo và khởi tạo:** định nghĩa timescale, chu kỳ clock, instantiate DUT và tạo waveform dump.
- **Clock generation:** tạo xung clock đối xứng chu kỳ $CYCLE = 100ns$.
- **Stimulus:** 6 khối initial lần lượt áp dụng các tổ hợp `rst_n` và `flick` để kiểm tra.

4.2 Cấu trúc Testbench

```
1  `timescale 1ns/1ps
2  `define CYCLE 100
3
4  module testbench;
5      reg          clk;
6      reg          rst_n;
7      reg          flick;
8      wire [15:0] lamp;
9      // Instantiate DUT
10     bound_flasher bound_flasher_01 (
11         .lamp (lamp),
12         .clk  (clk),
13         .rst_n (rst_n),
14         .flick (flick)
15     );
16     // Clock generation: period = CYCLE = 100ns
17     initial clk = 0;
18     always #('CYCLE/2) clk = ~clk;
19     // Waveform dump for SimVision
20     initial begin
21         $recordfile("waves");
22         $recordvars("depth=0", testbench);
23     end
24     // Signal monitor
25     initial begin
26         $monitor("time=%0d state=%0d flick=%b lamp=%h",
27             $time, testbench.bound_flasher_01.state,
28             flick, lamp);
29     end
30 endmodule
31 // TEST CASE 1: flick=0 throughout the entire sequence
32 rst_n = 0; flick = 0;
33 #('CYCLE*3);          // Hold reset for 3 cycles
34 rst_n = 1;
35 #('CYCLE*2);          // Wait 2 cycles after reset release
36 flick = 1;
37 #('CYCLE*1);          // Pulse flick for 1 cycle to start sequence
38 flick = 0;
39 #('CYCLE*100);         // Wait for full sequence to complete
40 // TEST CASE 2: kickback only at lamp[5] in S3
41 rst_n = 0; flick = 0;
```

```
42 #('CYCLE*3);           // Hold reset 3 cycles
43 rst_n = 1;
44 #('CYCLE*2);           // Wait 2 cycles after reset
45 flick = 1;
46 #('CYCLE*1);           // Pulse flick to start sequence
47 flick = 0;
48 // Wait until lamp[5] kickback point in S3:
49 #('CYCLE*18);
50 flick = 1;             // Assert flick at lamp[5] -> trigger kickback -> S3K
51 #('CYCLE*3);           // Hold flick long enough to be sampled
52 flick = 0;
53
54 // TEST CASE 3: flick=1 at lamp[10] kickback point in S3 (L0->L10)
55 rst_n = 0; flick = 0;
56 #('CYCLE*3);
57 rst_n = 1;
58 #('CYCLE*2);
59 flick = 1;
60 #('CYCLE*1);
61 flick = 0;
62 // S1(6) + S2(6) + S3 up to lamp[10](11) = 23 cycles
63 #('CYCLE*23);
64 flick = 1;             // Kickback at lamp[10] in S3
65 #('CYCLE*2);
66 flick = 0;
67 #('CYCLE*200);
68
69 // TEST CASE 4: flick=1 at lamp[10] kickback point in S5 (L5->L15)
70 rst_n = 0; flick = 0;
71 #('CYCLE*3);
72 rst_n = 1;
73 #('CYCLE*2);
74 flick = 1;
75 #('CYCLE*1);
76 flick = 0;
77 // S1(6) + S2(6) + S3(11) + S4(6) + S5 up to lamp[10](6) = 35 cycles
78 #('CYCLE*35);
79 flick = 1;             // Kickback at lamp[10] in S5
80 #('CYCLE*2);
81 flick = 0;
82 #('CYCLE*200);
83
84 // TEST CASE 5: flick=1 at both lamp[5] and lamp[10] kickback points in S3
85 rst_n = 0; flick = 0;
86 #('CYCLE*3);
87 rst_n = 1;
88 #('CYCLE*2);
89 flick = 1;
90 #('CYCLE*1);
91 flick = 0;
92 // S1(6) + S2(6) + S3 up to lamp[5](6) = 18 cycles
93 #('CYCLE*18);
94 flick = 1;             // Kickback #1 at lamp[5]
95 #('CYCLE*2);
96 flick = 0;
97 // S3K(6) + S3 lamp[0]->lamp[5](6) + lamp[5]->lamp[10](5) = 17 cycles
98 #('CYCLE*17);
```

```
99 flick = 1;           // Kickback #2 at lamp[10]
100 #('CYCLE*2);
101 flick = 0;
102 #('CYCLE*250);
103
104 // TEST CASE 6: flick=1 at all 3 kickback points
105 rst_n = 0; flick = 0;
106 #('CYCLE*3);
107 rst_n = 1;
108 #('CYCLE*2);
109 flick = 1;
110 #('CYCLE*1);
111 flick = 0;
112 // S1(6) + S2(6) + S3 up to lamp[5](6) = 18 cycles
113 #('CYCLE*18);
114 flick = 1;           // Kickback 1: lamp[5] in S3
115 #('CYCLE*2);
116 flick = 0;
117 // S3K(6) + S3 lamp[0]->lamp[10](11) = 17 cycles
118 #('CYCLE*17);
119 flick = 1;           // Kickback 2: lamp[10] in S3
120 #('CYCLE*2);
121 flick = 0;
122 // S3K(10) + S3(11) + S4(6) + S5 up to lamp[10](6) = 33 cycles
123 #('CYCLE*33);
124 flick = 1;           // Kickback 3: lamp[10] in S5
125 #('CYCLE*2);
126 flick = 0;
127 #('CYCLE*300);
128 $finish;
```

4.3 Mô tả các Test Case

TC	Tên test case	Mục tiêu kiểm tra
TC1	flick=0 toàn bộ quá trình	Kiểm tra luồng hoạt động chuẩn không kickback: INIT→S1→S2→S3→S4→S5→S6→INIT, tổng 62 cycles.
TC2	Kickback tại lamp[5] trong S3	Kiểm tra cơ chế kickback lần lượt tại cả hai điểm nút của S3.
TC3	Kickback tại lamp[10] trong S3	Kiểm tra kickback chỉ tại điểm cuối lamp[10] của S3, bỏ qua điểm lamp[5].
TC4	Kickback tại lamp[10] trong S5	Kiểm tra cơ chế kickback tại điểm nút duy nhất của S5.
TC5	Kickback tại lamp[5] và lamp[10] trong S3	Kiểm tra kickback liên tiếp hai lần trong cùng một lần chạy S3.
TC6	Kickback tại cả 3 điểm nút	Kiểm tra toàn bộ cơ chế kickback: lamp[5] trong S3, lamp[10] trong S3, và lamp[10] trong S5 trong một chu kỳ mô phỏng.

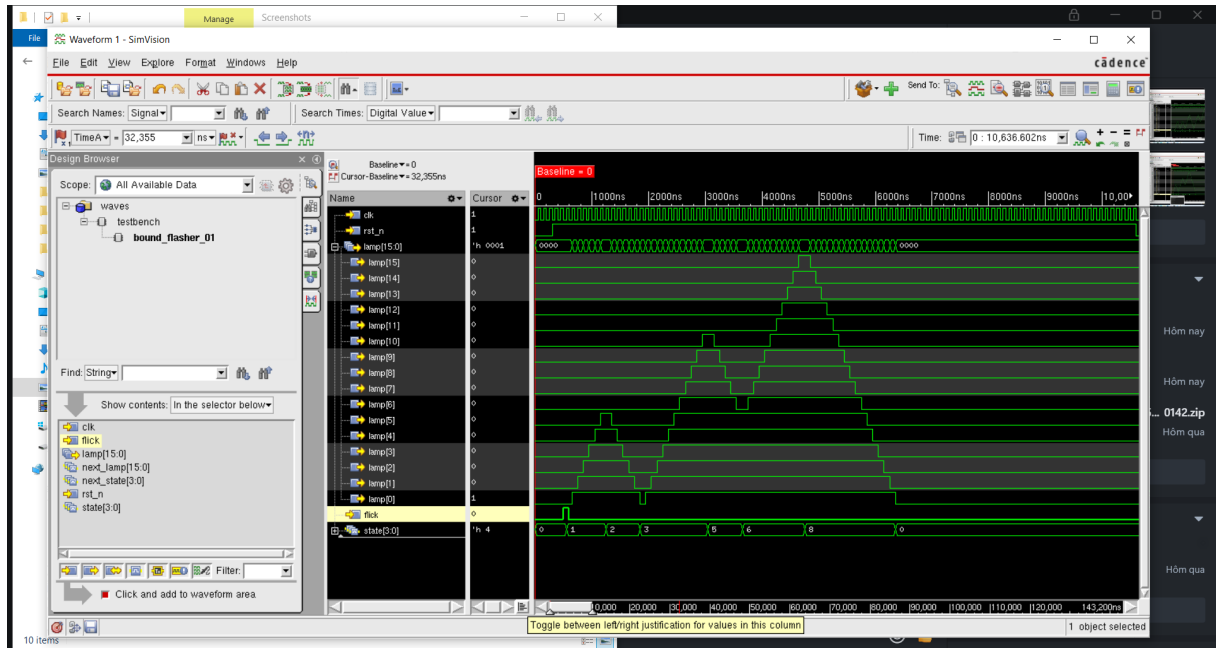
Bảng 8: Bảng mô tả các test case của testbench

4.4 Chi tiết từng Test Case

Test Case 1 - Luồng hoạt động chuẩn, không kickback

Mục tiêu: Xác minh toàn bộ chuỗi hoạt động bình thường khi flick = 0 tại mọi điểm kickback. Hệ thống phải đi qua đúng thứ tự INIT→S1→S2→S3→S4→S5→S6→INIT trong **62 clock cycles**.

INIT → S1(6) → S2(6) → S3(11) → S4(6) → S5(11) → S6(16) → INIT

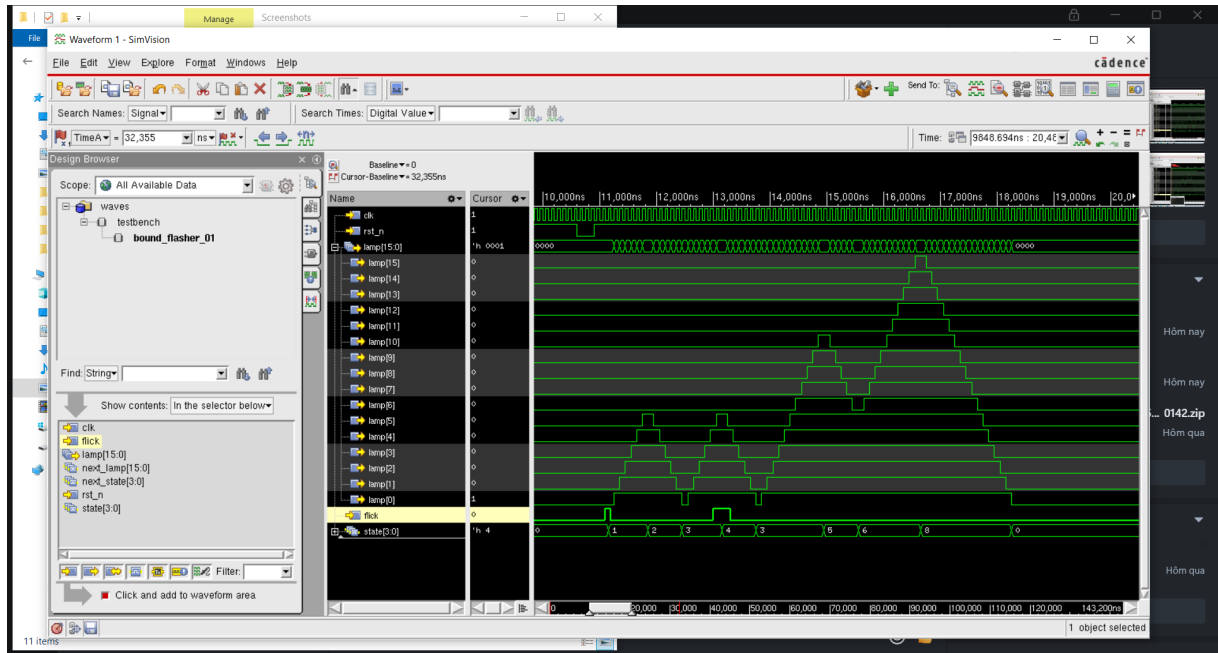


Hình 6: Waveform TC1 - Luồng hoạt động chuẩn không kickback

Test Case 2 - Kickback tại lamp[5] trong S3

Mục tiêu: Kiểm tra cơ chế kickback tại **điểm nút lamp[5]** trong giai đoạn S3. Khi flick = 1 tại thời điểm lamp[5] = 1, hệ thống chuyển sang trạng thái S3K, tắt dần toàn bộ đèn về lamp[0], sau đó khởi động lại S3 từ đầu và tiếp tục chuỗi hoạt động bình thường đến INIT.

INIT → S1(6) → S2(6) → S3 $\xrightarrow{\text{flick}=1 \text{ tại lamp}[5]}$ S3K → S3(11) → S4(6) → S5(11) → S6(16) → INIT

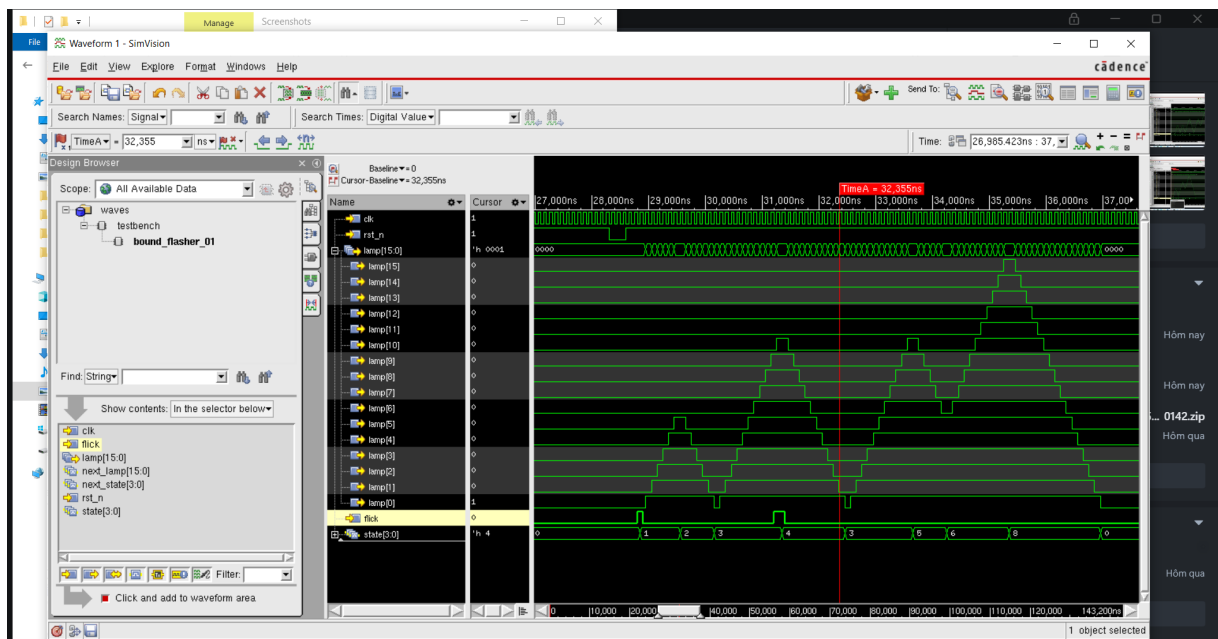


Hình 7: Waveform TC2 - Kickback tại lamp[5] trong S3

Test Case 3 - Kickback tại lamp[10] trong S3

Mục tiêu: Kiểm tra kickback chỉ tại điểm cuối lamp[10] của S3, bỏ qua điểm lamp[5]. Hệ thống vào S3K, tắt dần 11 bước về lamp[0] rồi khởi động lại S3 từ đầu.

INIT → S1(6) → S2(6) → S3 $\xrightarrow{\text{flick=1 tại lamp[10]}}$ S3K → S3(11) → S4(6) → S5(11) → S6(16) → INIT

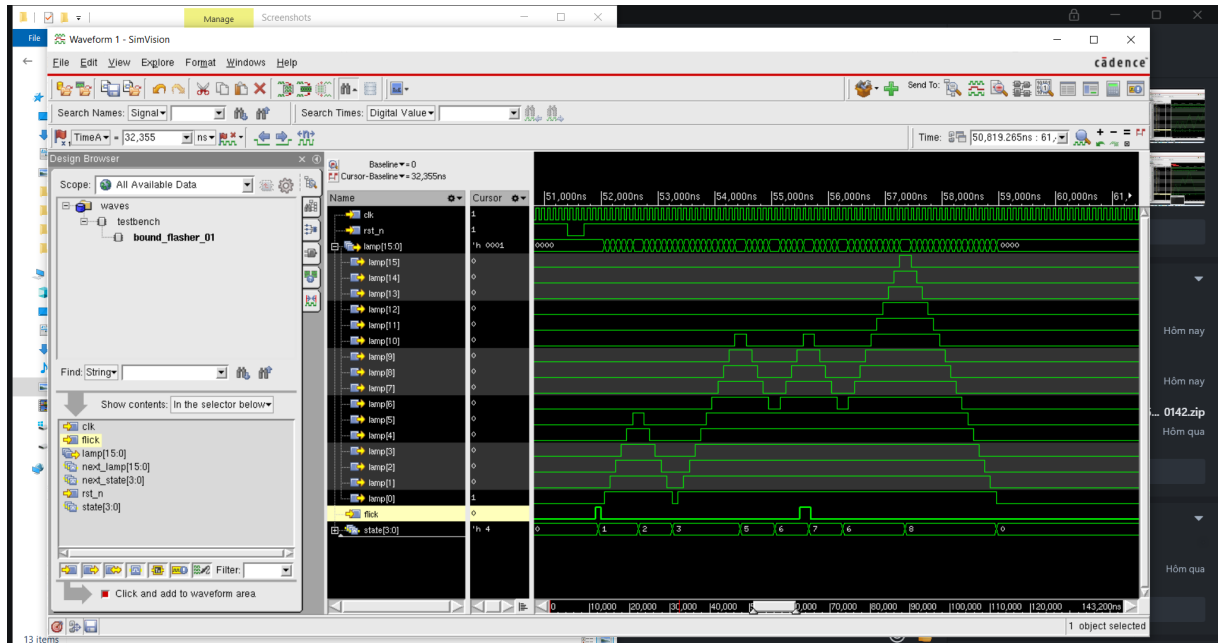


Hình 8: Waveform TC3 - Kickback tại lamp[10] trong S3

Test Case 4 - Kickback tại lamp[10] trong S5

Mục tiêu: Kiểm tra cơ chế kickback tại điểm nút duy nhất của S5. Khi flick = 1 tại lamp[10], hệ thống vào S5K, tắt dần về lamp[5] rồi khởi động lại S5.

INIT → S1(6) → S2(6) → S3(11) → S4(6) → S5 $\xrightarrow{\text{flick}=1 \text{ tại lamp}[10]}$ S5K → S5(11) → S6(16) → INIT

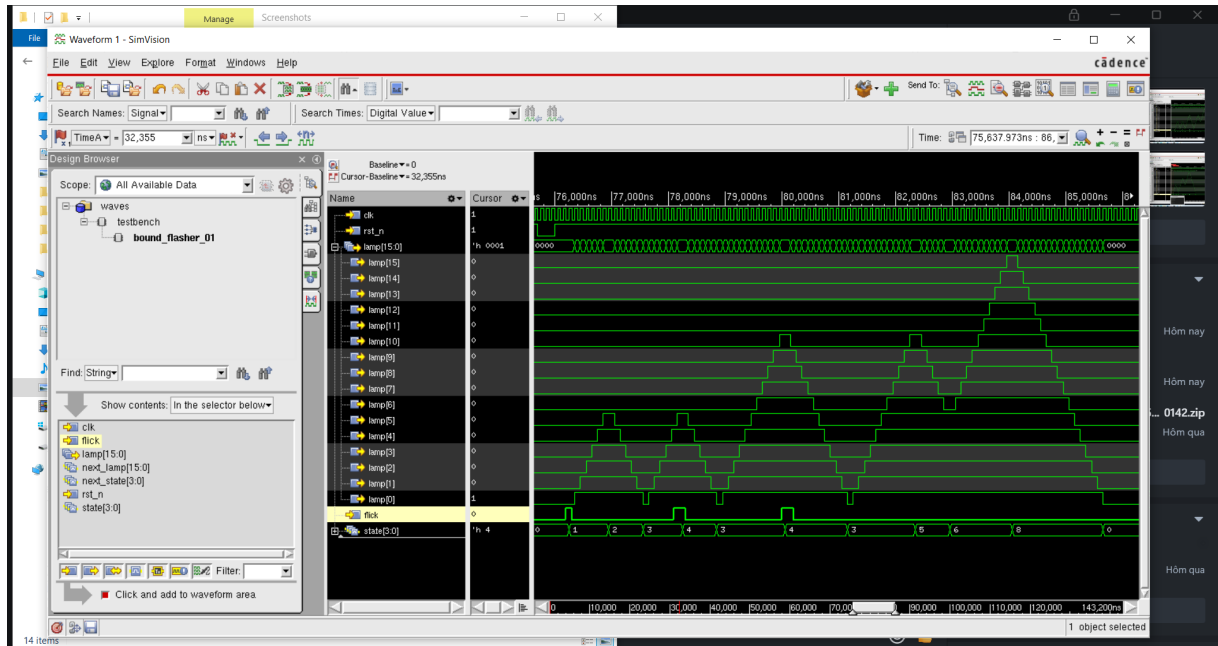


Hình 9: Waveform TC4 - Kickback tại lamp[10] trong S5

Test Case 5 - Kickback liên tiếp tại lamp[5] rồi lamp[10] trong S3

Mục tiêu: Kiểm tra khả năng kickback lặp lại trong cùng một giai đoạn S3 - đây là ràng buộc 3 của đặc tả. Sau kickback tại lamp[5], hệ thống quay lại S3 và kickback tiếp tại lamp[10].

INIT → S1(6) → S2(6) → S3 $\xrightarrow{\text{flick}=1 \text{ tại lamp}[5]}$ S3K → S3 $\xrightarrow{\text{flick}=1 \text{ tại lamp}[10]}$ S3K → S3(11) → S4(6) → S5(11) → S6(16) → INIT



Hình 10: Waveform TC5 - Kickback liên tiếp tại lamp[5] rồi lamp[10] trong S3

Test Case 6 - Kickback tại cả 3 điểm nút

Mục tiêu: Kiểm tra toàn bộ cơ chế kickback trong một chu kỳ mô phỏng duy nhất: kickback tại lamp[5] trong S3, kickback tại lamp[10] trong S3, và kickback tại lamp[10] trong S5. Đây là test case phức tạp nhất, bao phủ mọi nhánh điều kiện của thiết kế.

INIT → S1(6) → S2(6) → S3 $\xrightarrow{\text{flick=1 tại lamp[5]}}$ S3K → S3 $\xrightarrow{\text{flick=1 tại lamp[10]}}$ S3K → S3(11) → S4(6) → S5 $\xrightarrow{\text{flick=1 tại lamp[10]}}$ S5K → S5(11) → S6(16) → INIT



Hình 11: Waveform TC6 - Kickback tại cả 3 điểm nút

5 Kết luận

5.1 Tổng kết Lab

Lab 1 đã hoàn thành toàn bộ quy trình kiểm tra và xác minh thiết kế RTL cho module **Bound Flasher** trên nền tảng **Cadence Xcelium**, bao gồm các bước từ xây dựng môi trường mô phỏng, viết testbench đến phân tích waveform bằng **SimVision**.

5.2 Kết quả đạt được

TC	Kịch bản	Kết quả	Nhận xét
TC1	Luồng chuẩn, không kickback	PASS	Hệ thống đi đúng thứ tự INIT→S1→S2→S3→S4→S5→S6→INIT trong 62 clock cycles.
TC2	Kickback tại lamp[5] trong S3	PASS	Hệ thống vào S3K đúng thời điểm, tắt dần về lamp[0] và khởi động lại S3.
TC3	Kickback tại lamp[10] trong S3	PASS	Kickback tại điểm cuối S3 hoạt động chính xác, tắt dần 11 bước về lamp[0].
TC4	Kickback tại lamp[10] trong S5	PASS	Hệ thống vào S5K, tắt dần về lamp[5] và khởi động lại S5 đúng đặc tả.
TC5	Kickback liên tiếp tại lamp[5] rồi lamp[10] trong S3	PASS	Kickback lặp lại hai lần trong cùng giai đoạn S3, xác nhận ràng buộc 3 của đặc tả.
TC6	Kickback tại cả 3 điểm nút	PASS	Toàn bộ cơ chế kickback hoạt động đúng trong một chu kỳ mô phỏng duy nhất.

Bảng 9: Tổng hợp kết quả các test case

5.3 Những điểm đã học được

- **Xây dựng môi trường mô phỏng chuẩn công nghiệp:** Nắm vững quy trình thiết lập thư mục, cấu hình license và biên dịch thiết kế bằng lệnh xrun trên **Cadence Xcelium**.
- **Thiết kế FSM theo kiến trúc Two-Always:** Hiểu rõ nguyên tắc tách biệt khối tuần tự và khối tổ hợp, tránh các lỗi phổ biến như *latch inference* và *race condition* trong thiết kế RTL.
- **Viết testbench có hệ thống:** Xây dựng 6 test case bao phủ đầy đủ các kịch bản hoạt động bình thường và ngoại lệ, bao gồm cả các trường hợp kickback lặp nhiều lần.
- **Phân tích waveform bằng SimVision:** Sử dụng thành thạo các công cụ *Source Browser*, *Schematic Tracer* và các tính năng zoom để debug và xác minh hành vi thiết kế.
- **Tư duy kiểm tra có hệ thống:** Hình thành thói quen phân tích đặc tả, xác định các điểm nút quan trọng và xây dựng test case bao phủ toàn diện trước khi chạy mô phỏng.